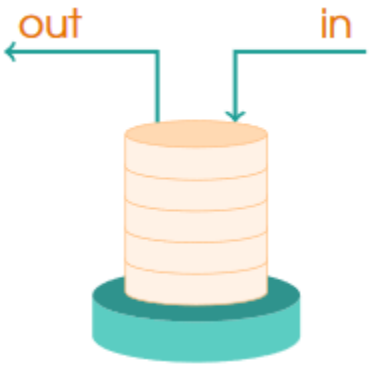
Structures de données linéaires. Les piles. Correction.

En informatique, une pile (en anglais stack) est une structure de données fondée sur le principe « dernier arrivé, premier sorti » (ou LIFO pour Last In, First Out), ce qui veut dire que les derniers éléments ajoutés à la pile seront les premiers à être récupérés.
Le fonctionnement est donc celui d'une pile d'assiettes : on ajoute des assiettes sur la pile, et on les récupère dans l'ordre inverse, en commençant par la dernière ajoutée.



Voici quelques exemples d'usage courant d'une pile :
• Dans un navigateur web, une pile sert à mémoriser les pages Web visitées. L'adresse de chaque nouvelle page visitée est empilée et l'utilisateur dépile l'adresse de la page précédente en cliquant le bouton « Afficher la page précédente ».
• L'évaluation des expressions mathématiques en notation post-fixée (ou polonaise inverse) utilise une pile.
• La fonction « Annuler la frappe » (en anglais « Undo ») d'un traitement de texte mémorise les modifications apportées au texte dans une pile.

Pour implémenter une structure de pile, on a besoin d'un nombre réduit d'opérations de bases :
• « empiler » : ajoute un élément sur la pile. Terme anglais correspondant : « Push ».
• « dépiler » : enlève un élément de la pile et le renvoie. En anglais : « Pop ».
• « vide » : renvoie vrai si la pile est vide, faux sinon.
• « remplissage » : renvoie le nombre d'éléments dans la pile.

La structure de pile est un concept abstrait. Comment la réaliser dans la pratique ?
Voici plusieurs implémentations possibles. L'idée principale étant que les fonctions de bases pourront être utilisées indépendamment de l'implémentation choisie.

I) IMPLEMENTATION - METHODE 1 :

Nous utiliserons une simple liste pour représenter la pile.
Il se trouve que les méthodes append et pop sur les listes jouent déjà le rôle de push et pop sur les piles.
Voici les fonctions de base :

```
def pile():
    #retourne une liste vide
    return []

# vide
def vide(p):
    '''renvoie True si la pile est vide
    et False sinon'''
    return p==[]

# empiler
```

```
def empiler(p,x):
    "Ajoute l'élément x à la pile p"
    p.append(x)

# dépiler
def depiler(p):
    "dépile et renvoie l'élément au sommet de la pile p"
    assert not vide(p), "Pile vide"
    return p.pop()
```

À faire 1 :
Tester les instructions suivantes :

```
p=pile()
for i in range(5):
    empiler(p,2*i)
a=depiler(p)
print(a)
print(vide(p))
```

Correction :

```
p=pile()
```

Création d'une pile vide p.

```
for i in range(5):
    empiler(p,2*i)
```

Remplissage de la pile par : [0, 2, 4, 6, 8]

```
a=depiler(p)
```

On retire le dernier élément de la pile. Il reste : [0, 2, 4, 6]

```
print(a)
```

Retourne l'élément qui vient d'être dépilé : 8

```
print(vide(p))
```

Retourne False (la liste n'est pas vide).

Exercice 1 :
Réaliser les fonctions taille(p) et sommet(p) qui retournent respectivement la taille de la liste et le sommet de la liste (sans le supprimer).

```
def taille(p):
    pass

def sommet(p):
    pass
```

Correction :

```
def taille(p):
    nbre_elts = 0
    for i in p:
        nbre_elts += 1
    return nbre_elts
    # ou plus rapide return len(p)
print(taille(p))
```

```
def sommet(p):
    return p[-1]
```

```
print(sommet(p))
```

ou

```
def sommet(p):
    return p[taille(p)-1]
```

II) IMPLEMENTATION - METHODE 2 :

Nous allons créer une classe Pile pour implémenter cette structure.
Voici la classe :

```
class Pile:
    ''' classe Pile
    création d'une instance Pile avec une liste
    '''
    def __init__(self):
        "Initialisation d'une pile vide"
        self.L=[]

    def vide(self):
        "teste si la pile est vide"
        return self.L==[]

    def depiler(self):
        "dépile"
        assert not self.vide(),"Pile vide"
        return self.L.pop()

    def empiler(self,x):
        "empile"
        self.L.append(x)
```

À faire 2 :
Tester les instructions suivantes :

```
p=Pile()
for i in range(5):
    p.empiler(2*i)
print(p.L)
a=p.depiler()
print(a)
print(p.L)
print(p.vide())
```

Correction :

```
On obtient les mêmes résultats que précédemment :
[0, 2, 4, 6, 8]
8
[0, 2, 4, 6]
False
```

Exercice 2 :
Réaliser les méthodes taille(self) et sommet(self) qui retournent respectivement la taille de la liste et le sommet de la liste (sans le supprimer).

```
def taille(self):
    pass

def sommet(self):
    pass
```

Correction :

```
Dans la classe Pile :
def taille(self):
        return len(self.L)

    def sommet(self):
        return self.L[-1]

# en dehors de la classe :
print(p.taille())
print(p.sommet())
```

III) UN PREMIER EXEMPLE - CONTROLE DU PARENTHESAGE D'UNE EXPRESSION :

Il s'agit d'écrire une fonction qui contrôle si une expression mathématique, donnée sous forme d'une chaîne de caractères, est bien parenthésée, c'est- à-dire s'il y a autant de parenthèses ouvrantes que de fermantes, et qu'elles sont bien placées.
Par exemple :
(..(..)..) est bien parenthésée.
(...(..(..)...) ne l'est pas.

L'algorithme :
On crée une pile.
On parcourt l'expression de gauche à droite.
À chaque fois que l'on rencontre une parenthèse ouvrante "(" on l'empile.
Si on rencontre une parenthèse fermante ") " et que la pile n'est pas vide on dépile (sinon on retourne faux).
À la fin la pile doit être vide...

À faire 3 :
Écrire en pseudo code l'algorithme.
Algorithme.
Entrée : une expression en écriture habituelle (une chaîne de caractères).
Sortie : « vrai » si les parenthèses sont cohérentes, « faux » sinon.
• Partir avec une pile vide.
• Pour chaque caractère de l'expression lue de gauche à droite :
— si le caractère n'est ni "(", ni ")" alors ne rien faire !
— si le caractère est une parenthèse ouvrante "(" alors ajouter ce caractère à la pile ;
— si le caractère est une parenthèse fermante ")" :
— tester si la pile est vide, si elle vide alors renvoyer « faux » (le programme se termine là, l'expression est mal parenthésée), si la pile n'est pas vide continuer,
— dépiler une fois, on dépile un "(".
• Si à la fin, la pile est vide alors renvoyer la valeur « vrai », sinon renvoyer « faux ».

Voici l'algorithme qui décide si les parenthèses d'une expression sont bien placées. La pile joue le rôle d'une zone de stockage intermédiaire pour les parenthèses ouvrantes "(". Chaque fois que l'on trouve une parenthèse fermante ")" dans l'expression on supprime une parenthèse ouvrante de la pile.

Exercice 3 :
En utilisant l'une des structures pile réalisées plus haut, écrire une fonction verification(expr) qui vérifie si une expression mathématique passée en paramètre est correctement parenthésée.
Correction :

```
class Pile:
    ''' classe Pile
    création d'une instance Pile avec une liste
    '''
    def __init__(self):
        "Initialisation d'une pile vide"
        self.L=[]
```

```
    def vide(self):
        "teste si la pile est vide"
        return self.L==[]

    def depiler(self):
        "dépile"
        assert not self.vide(),"Pile vide"
        return self.L.pop()

    def empiler(self,x):
        "empile"
        self.L.append(x)

    def taille(self):
        return len(self.L)

    def sommet(self):
        return self.L[-1]

def verification(expr):
    for i in expr:
        if i == '(':
            p.empiler('(')
        if i == ')':
            if p.vide():
                return 'Faux'
            else:
                p.depiler()
    if p.vide():
        return 'Vrai'
    else:
        return 'Faux'

p=Pile()
print(verification('((gdg(g)g)fg)'))
```

Retourne : Vrai

<u>Exercice 4</u> :
Faire en sorte que le programme tienne compte également des [.
Correction :
Algorithme :
Entrée : une expression en écriture habituelle (une chaîne de caractères).
Sortie : « vrai » si les parenthèses et les crochets sont cohérents, « faux » sinon.
• Partir avec une pile vide.
• Pour chaque caractère de l'expression lue de gauche à droite :
— si le caractère n'est ni "(", ni ")", ni "[", ni "]" alors ne rien faire ;
— si le caractère est une parenthèse ou un crochet ouvrant "(" ou "[", alors ajouter ce caractère à la pile ;
— si le caractère est une parenthèse ou un crochet fermant ")" ou "]" :
— tester si la pile est vide, si elle est vide alors renvoyer « faux » (le programme se termine là, l'expression n'est pas cohérente), si la pile n'est pas vide continuer,
— dépiler une fois, on dépile un "(" ou un "[",
— si le caractère dépilé (ouvrant) ne correspond pas au caractère lu dans l'expression, alors renvoyer « faux ». Le programme se termine là, l'expression n'est pas cohérente ; dire que les caractères correspondent c'est avoir "(" avec ")" et "[" avec "]".
• Si à la fin, la pile est vide alors renvoyer la valeur « vrai », sinon renvoyer « faux ».

Cette fois la pile peut contenir des parenthèses ouvrantes "(" ou biens des crochets ouvrants "[". Chaque fois que l'on trouve une parenthèse fermante ")" dans l'expression, il faut que le haut de la pile soit une parenthèse ouvrante "(".

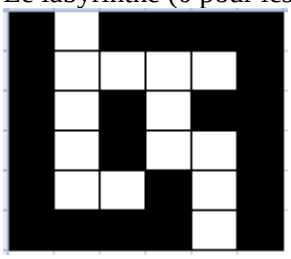
Chaque fois que l'on trouve un crochet fermant "]" dans l'expression, il faut que le haut de la pile soit un crochet ouvrant "[".

```
def verification(expr):
    for i in expr:
        if i == '(' or i == '[':
            p.empiler(i)
        if i == ')' or i == ']':
            if p.vide():
                return 'Faux'
            elif (i ==')' and p.sommet() != '(') or (i == ']' and p.sommet() != '['):
                return 'Faux'
            else:
                p.depiler()
    if p.vide():
        return 'Vrai'
    else:
        return 'Faux'
```

Prolongement possible : on pourrait imaginer pour un projet de développer davantage cette vérification jusqu'à proposer de montrer l'erreur, voire d'en proposer une correction.

IV) UN SECOND EXEMPLE - RESOUDRE UN LABYRINTHE :

On représentera le labyrinthe à l'aide d'un tableau (liste de listes).
Le labyrinthe (0 pour les murs noirs), (1 pour les murs blancs).



laby=[[0,1,0,0,0,0],
[0,1,1,1,1,0],
[0,1,0,1,0,0],
[0,1,0,1,1,0],
[0,1,1,0,1,0],
[0,0,0,0,1,0]]

- On repère une case par ses coordonnées (*i;j*), on y accède dans le tableau par laby[i][j].
- L'entrée se fait par la case (0;1) et la sortie par la case (5;4).

Question 1 :
Compléter le code pour obtenir le nombre de lignes et le nombre de colonnes de ce tableau :

```
lignes=len( )
colonnes=len( )
```

Correction :

```
lignes=len(laby)
colonnes=len(laby[0])
```

Nous aurons besoin de créer une copie de ce tableau pour pouvoir travailler dessus sans altérer le tableau laby.
Nous utiliserons une bibliothèque pour cela.

À faire 4 :
Tester ce code

```
from copy import deepcopy
```

```
laby=[[0,1,0,0,0,0],
[0,1,1,1,1,0],
[0,1,0,1,0,0],
[0,1,0,1,1,0],
[0,1,1,0,1,0],
[0,0,0,0,1,0]]

T=deepcopy(laby)
T[3][2]='hello'

for ligne in laby:
    print(ligne)
print("----------------")
for ligne in T:
    print(ligne)
```

Ce code permet de faire une copie de laby, appelée T. Puis de mettre hello en ligne n° 3 (donc quatrième ligne), et en colonne n° 2 (donc troisième colonne). Enfin il affiche laby original, puis T.

Objectif : l'objectif est d'écrire un programme qui détermine s'il existe un chemin de l'entrée vers la sortie en se déplaçant vers le haut, le bas, la gauche ou la droite (mais pas en diagonale).

Étape 1 :
À faire 5 :

Voici une fonction qui prend en paramètre un tableau T et un tuple v

```
def voisins(T,v):
    V=[]
    i,j=v[0],v[1]
    for a in (-1,1):
        if 0<=i+a<lignes:
            if T[i+a][j]==1:
                V.append((i+a,j))
            if 0<=j+a<colonnes:
                if T[i][j+a]==1:
                    V.append((i,j+a))
    return V
```

Question 2 :
Que retourne cette fonction ?
Cette fonction renvoie une liste de tuples V[(a,b),(c,d),…]. Chaque tuple correspond aux coordonnées des cases blanches voisines (donc aux cases voisines accessibles) de la case v(i,j) passée en argument.

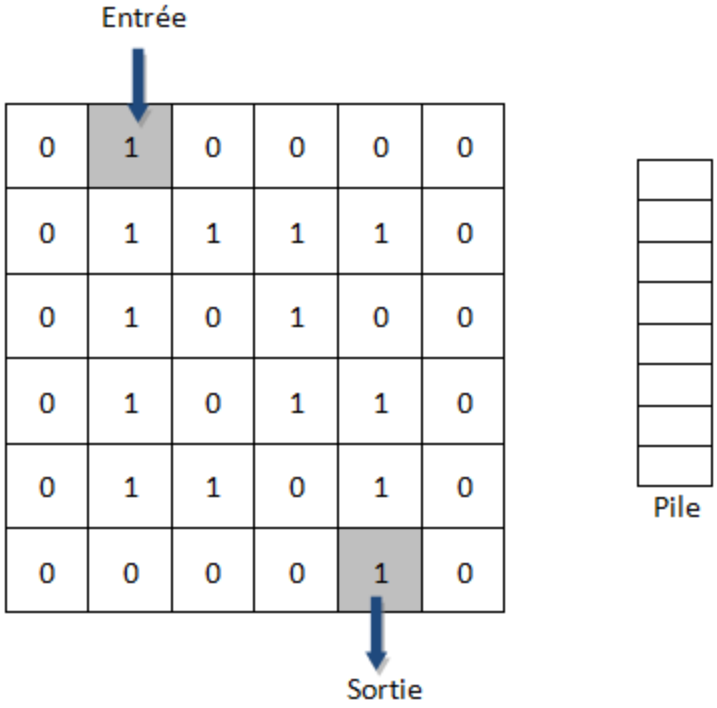
Question 3 :
Expliquer l'affichage provoqué par cette instruction : print(voisins(laby,(0,1)).
Cette instruction renvoie [(1, 1)]. En effet c'est la seule case blanche voisine de la case (0,1).
Si on essaye avec (1,1), on obtient [(0, 1), (2, 1), (1, 2)] (coordonnées des trois cases blanches voisines de (1,1).

Spécifier la fonction.

```
def voisins(T,v):
    """
    fonction qui renvoie une liste V de tuples composés des coordonnées
    des cases blanches voisines de v passé en argument pour le laryrinthe T
    """
….
```

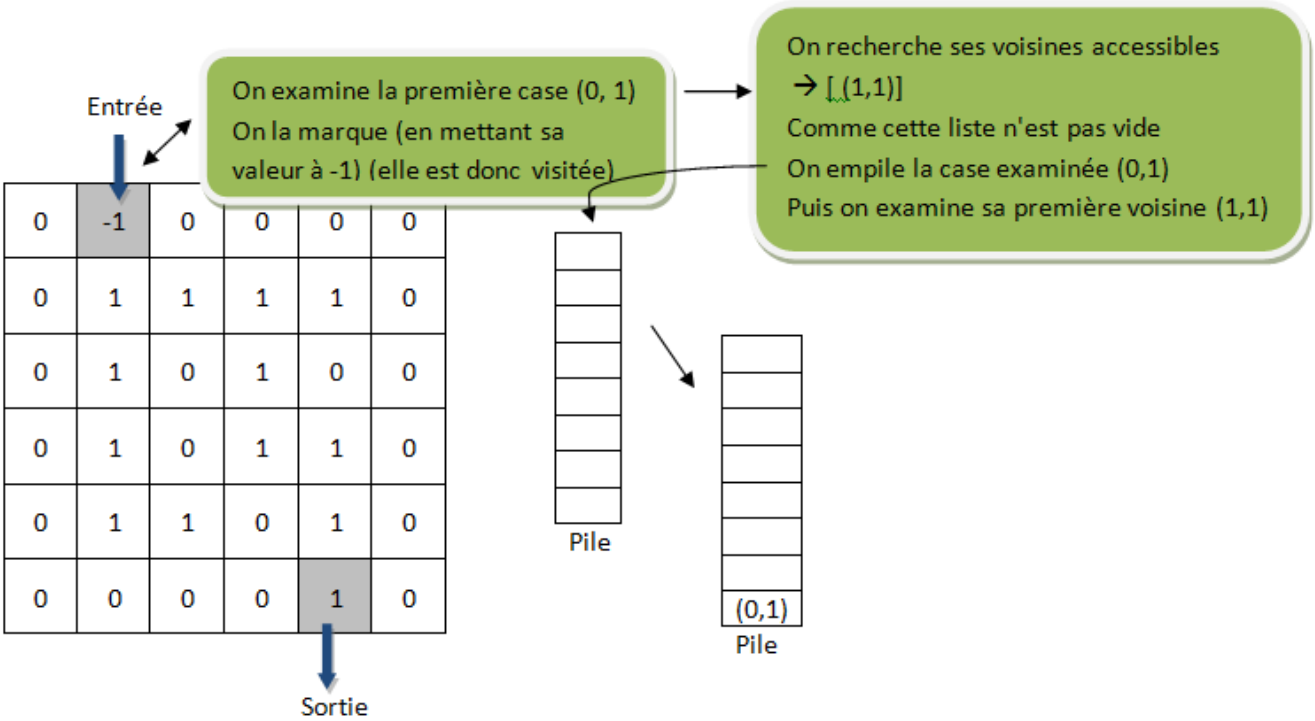
Étape 2 : parcours du labyrinthe
L'idée est de parcourir le labyrinthe depuis l'entrée, en utilisant une pile pour stocker le chemin, pour pouvoir dépiler lorsque le chemin n'aboutit pas et redémarrer sur une autre voie.

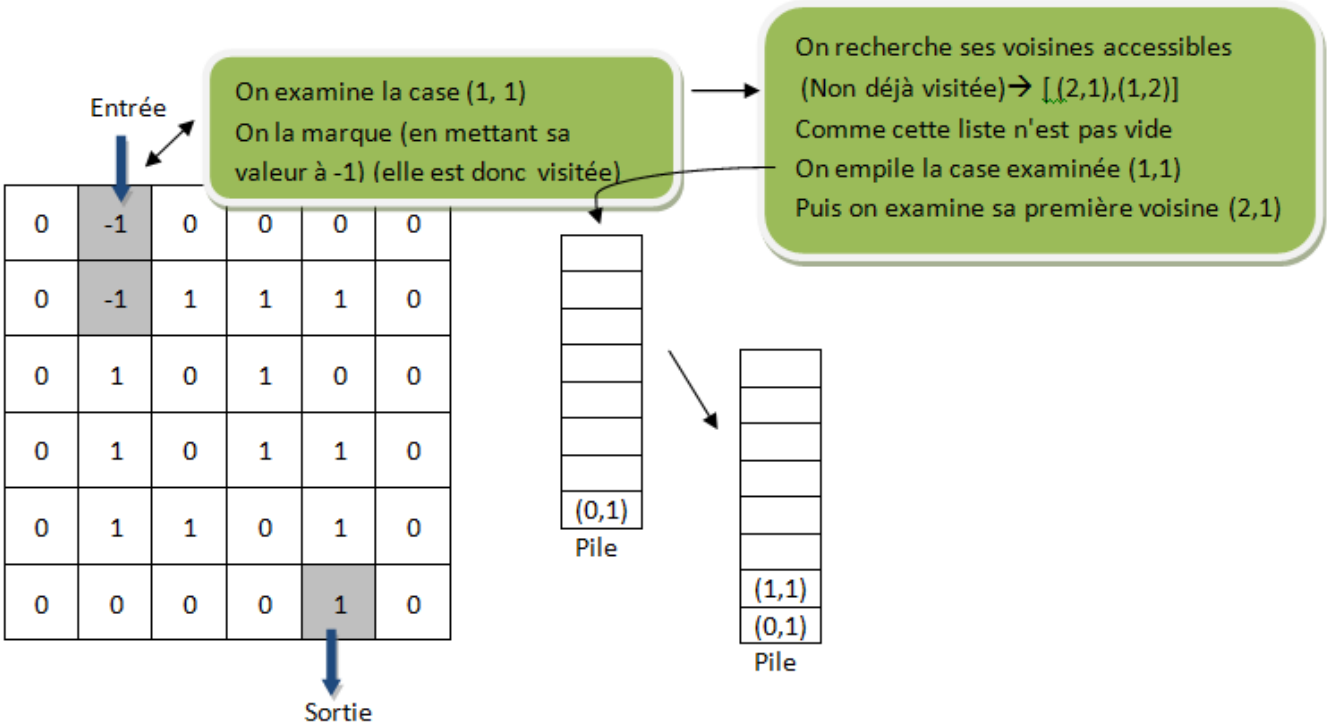
Un schéma sera sans doute plus efficace qu'un long discours...



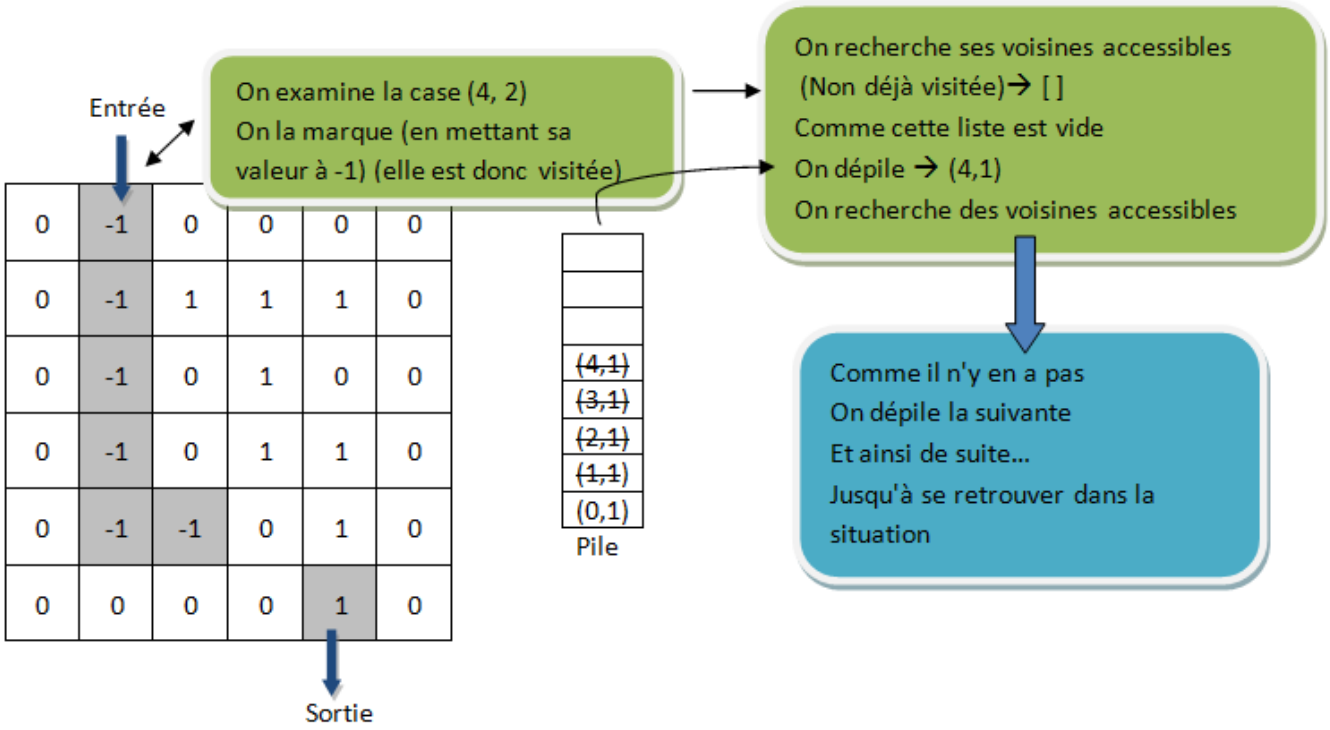
On dispose d'une copie de notre tableau et d'une pile vide.

<u>Le processus</u>.

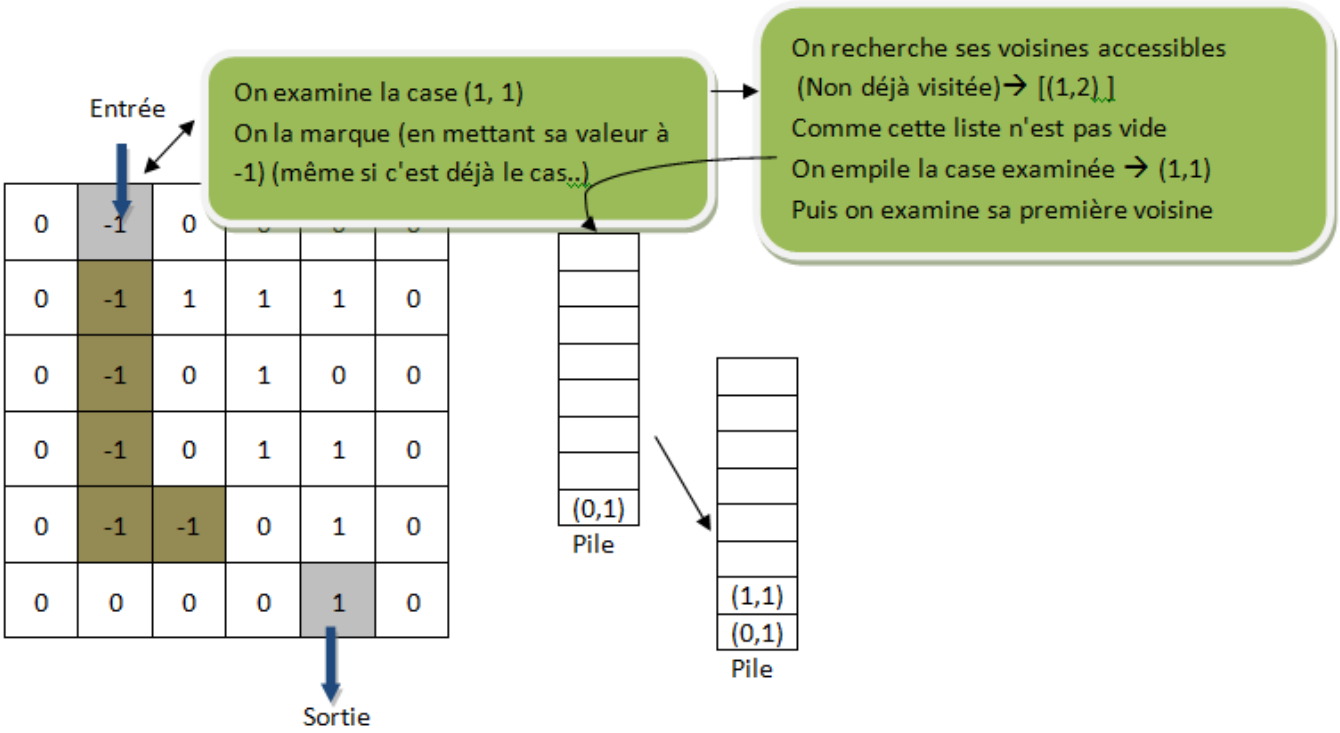




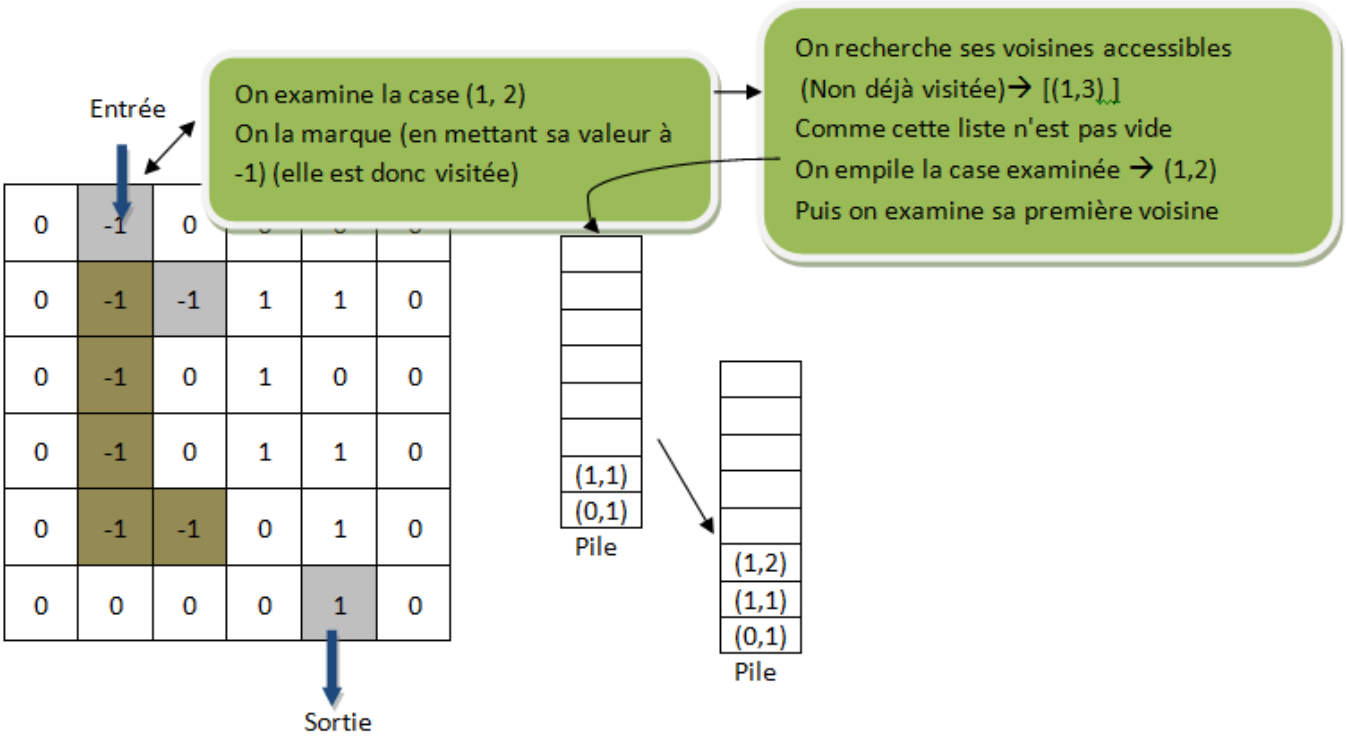
Et ainsi de suite jusqu'à ce que l'on tombe sur une impasse.



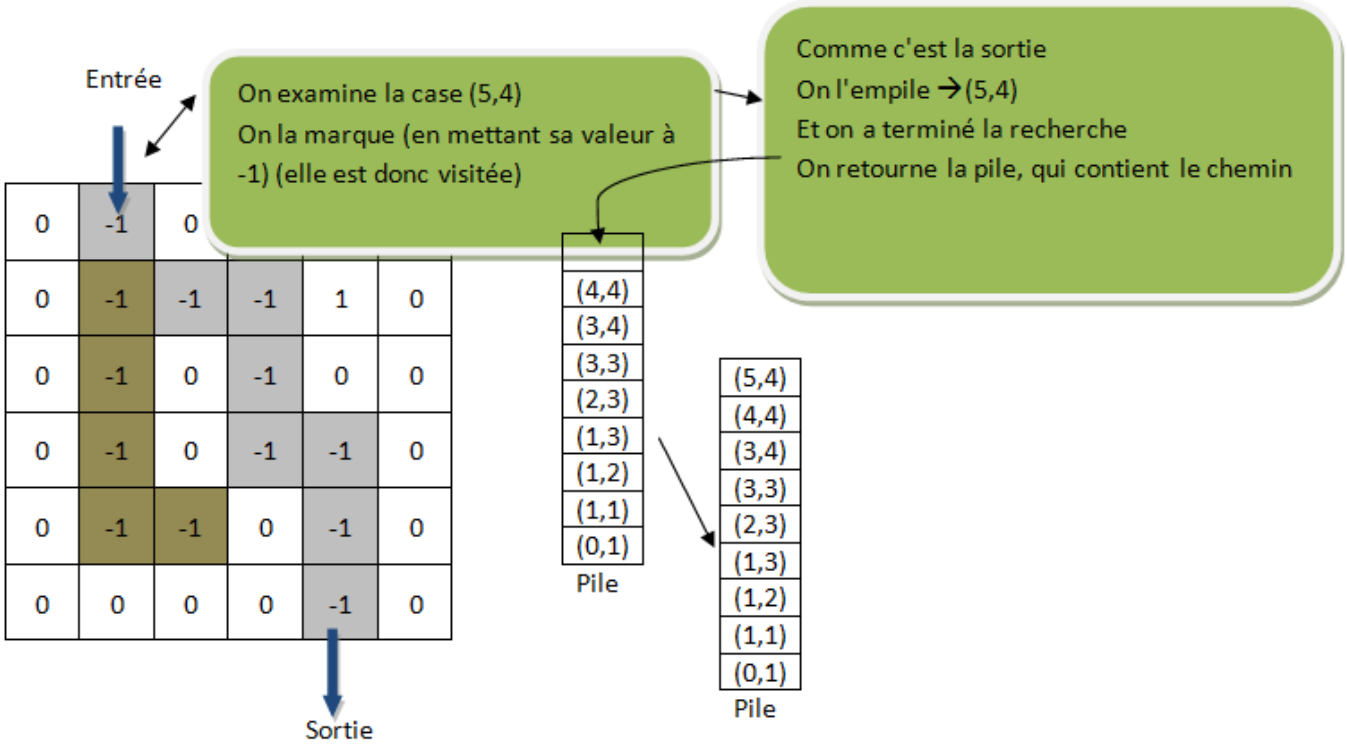
On examine la dernière case dépilée.



On poursuit l'exploration.



Et ainsi de suite jusqu'à la sortie.



L'algorithme :
À faire 6 :
Voici l'algorithme :

```
fonction parcours( laby, entree, sortie)
   T <-- une copie de laby
   p <-- Pile()
   v <-- entree
   on met à -1 la valeur de la case v dans T
   recherche <--True
   tant que recherche est vrai
      vois <-- voisins (T, v)
      si la liste vois est vide
          si la pile p est vide --> renvoyer False
          sinon v <--premier élément de la pile p (dernier élément de la « liste p ») et l'enlève
      sinon
         on empile p avec v
         v <-- le 1er élément de la liste vois
         on met à -1 la valeur de la case v dans T
         si v = sortie
            On dépile p avec sortie
            recherche <-- False
   return p
```

Question 4 :
Si au cours de l'exécution la pile se trouve vide, que cela signifie-t-il ?
Cela signifie que le labyrinthe n'a pas de solution. Il est impossible d'en sortir.

Question 5 :
Réaliser le programme (mettre en évidence le chemin) et tester le avec d'autres labyrinthes.

```
from copy import deepcopy
laby=[[0,1,0,0,0,0],
[0,1,1,1,1,0],
[0,1,0,1,0,0],
[0,1,0,1,1,0],
[0,1,1,0,1,0],
```

```
[0,0,0,0,1,0]]

lignes=len(laby)
colonnes=len(laby[0])

def voisins(T,v):
    """
    fonction qui renvoie une liste V de tuples composés des coordonnées
    des cases blanches voisines de v passé en argument pour le laryrinthe T
    """
    V=[]
    i,j=v[0],v[1]
    for a in (-1,1):
        if 0<=i+a<lignes:
            if T[i+a][j]==1:
                V.append((i+a,j))
            if 0<=j+a<colonnes:
                if T[i][j+a]==1:
                    V.append((i,j+a))
    return V

def parcours(laby,entree,sortie):
    T = deepcopy(laby)
    p = []
    v = entree
    T[v[0]][v[1]] = -1
    recherche = True
    while recherche:
        vois = voisins(T,v)
        if vois == []:
            if p == []:
                return False
            else:
                v = p.pop()

        else:
            p.append(v)
            v = vois[0]
            T[v[0]][v[1]] = -1
            if v == sortie:
                p.append(sortie)
                recherche = False
    return p

print(parcours(laby,(0,1),(5,4)))
```

Prolongement possible : pour un projet on pourrait imaginer un programme qui génère un labyrinthe et qui le résout...